Module 12: Final Project - AI Engineering Workbench

Complete system, evaluation report, and demo

Module 12: Final Project - AI Engineering Workbench

Primary sources: Course synthesis from Osmani; Thomas & Hunt, Chapter 9: Pragmatic Projects.

Learning Outcomes

- Design and implement a complete AI-assisted system.
- Integrate workflows, evaluation, and safeguards.
- Justify architectural and engineering decisions.
- Present and defend the system.

Final System Expectations

Your AI Engineering Workbench should demonstrate:

- Structured input handling.
- AI-assisted generation.
- Evaluation and testing.
- Controlled workflow management.
- Persistence or artifact tracking.
- Documentation.
- Human oversight.

- Collaborative engineering practice.

The final product is not just an app. It is an engineering system that shows how AI work is requested, constrained, evaluated, and accepted.

In this course, the AI Engineering Workbench is your controlled coding agent system plus its evaluation, safeguards, artifacts, documentation, and team process.

Workbench Architecture

Present the system as connected parts:

- User or developer input.
- Prompting and context assembly.
- AI interaction or agent execution.
- Generated artifact or proposed change.
- Automated checks and evaluation.
- Human review and decision point.
- Logging, persistence, or traceability.
- Documentation and handoff.

Every major component should have a reason to exist.

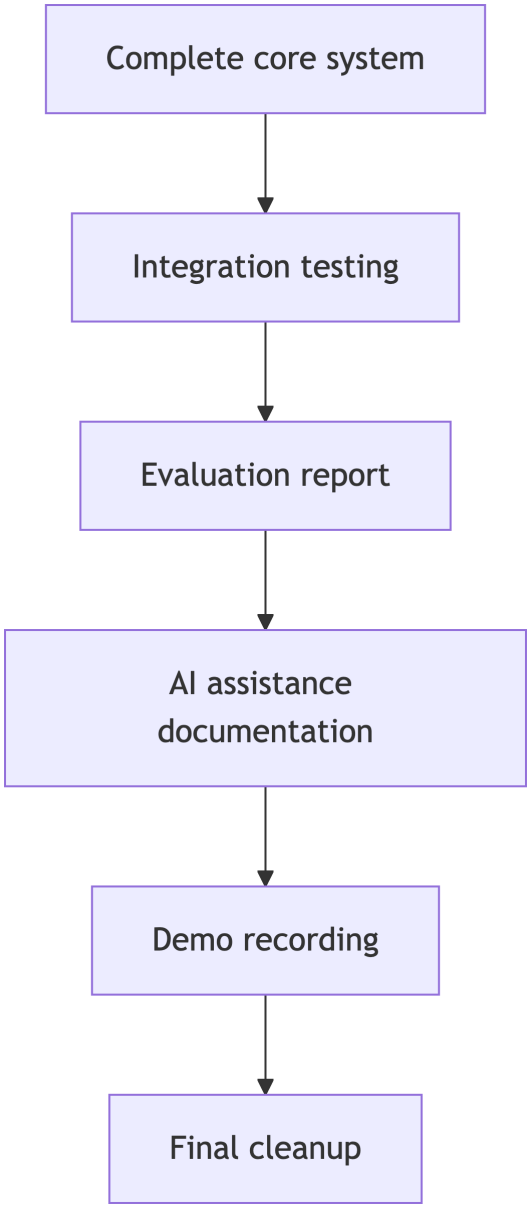
Integrated Workflow

A complete workflow should make the path visible:

1. A user or developer states a goal.
2. The system prepares context and constraints.
3. AI produces an output or action plan.
4. The system checks the result.
5. A human reviews important tradeoffs.
6. The result is accepted, revised, or rejected.
7. Evidence is saved for later inspection.

If your workflow skips one of these steps, be ready to justify why.

Final Submission Timeline



Reproducibility

A grader should be able to answer:

- How do I set it up?
- How do I run it?
- What example should I try?
- What output should I expect?
- How do I run tests?
- Where are evaluation results?
- Where is AI assistance documented?

Evaluation Report

The evaluation report should include:

- What was evaluated.
- Metrics or criteria.
- Test cases or scenarios.
- Results.
- Interpretation.
- Known limitations.
- What would improve next.

Strong reports connect results to claims. If you claim the system is reliable, secure, useful, or maintainable, show evidence that supports that claim.

Safeguards

Your final system should show explicit controls:

- Scope limits for what AI may change or generate.
- Input validation and output validation.
- Tests for expected behavior and edge cases.
- Review gates before irreversible actions.
- Logging of prompts, outputs, decisions, or artifacts.
- Handling for failed AI calls or low-quality output.
- Privacy and secret-handling practices.

Safeguards should be demonstrated, not only described.

Decision Justification

For each major decision, explain:

- The problem it solves.
- Alternatives considered.
- Why your choice fits the project constraints.
- What tradeoff it creates.
- How you know it worked well enough.

Good defense connects architecture to evidence.

AI Assistance Section

Your README should make AI use transparent:

- Planning.
- Code generation.
- Test generation.
- Debugging.
- Review.
- Integration support.
- Documentation support.
- Accepted, modified, and rejected suggestions.
- Verification methods.

Demo Structure

Recommended 5-8 minute flow:

1. Problem and users.
2. Architecture overview.
3. Live or recorded workflow run.
4. Evaluation results.
5. Safeguards and human oversight.
6. Team process and AI evidence.
7. Lessons learned.

Professional Defense

Be ready to explain:

- Why this architecture?
- What did AI help with?
- What did humans correct?
- What risks remain?
- Which tests matter most?
- What would you do next with more time?
- What would you not trust the system to do yet?
- Where did the team deliberately keep a human in the loop?

Final Quality Checklist

- Clean repository organization.
- No committed secrets.
- Fresh-environment setup tested.
- Tests and examples run.
- Demo link included.
- Required document paths submitted.
- Commit hash captured.
- Individual contributions clear.
- Architecture and workflow diagrams current.
- Safeguards exercised in the demo or report.

Course Synthesis

Responsible AI-assisted engineering means:

- Clear intent.
- Structured prompts.
- Human review.
- Automated evidence.
- Risk awareness.
- Transparent documentation.
- Team accountability.

The final project should make those practices concrete. A reviewer should be able to see not only what you built, but how your engineering process controlled AI's contribution.

Presentation Standard

Your presentation should defend the system, not narrate the repository.

- Start with the engineering problem.
- Show the workflow end to end.
- Explain why the architecture is appropriate.
- Demonstrate evaluation evidence.
- Identify safeguards and remaining risks.
- Close with what you learned and what should change next.

Prioritize clarity over feature count.

Closing Reflection

Submit an individual reflection tied to project evidence:

- What AI interaction changed your process most?
- What risk did you underestimate early?
- What verification habit will you keep?
- What work should remain human-led?

Reference one prompt, one test or evaluation result, and one rejected or revised AI suggestion.

Takeaways

- The final project is a demonstration of engineering judgment.
- AI assistance is strongest when controlled, evaluated, and documented.
- The goal is not to show that AI wrote code; it is to show that your team built trustworthy software with AI assistance.